

GUIDE-002：项目方法论总纲

万能产品小助手（PM AI）

魏子政审阅

2026-05-06 / 版本 v1.0

提出日期	2026-05-06
提出者	魏子政
优先级	P0
状态	已完成
文档类型	项目方法论（永久维护）

文档定位

本文档是项目的方法论总纲，回答以下核心问题：

1. 两种智能体（OpenClaw + Cursor）的选型和搭配细节是什么？
2. 属于 OpenClaw 的 Skill 和属于 Cursor 的 Skill 分别怎么加载？有何区别？
3. 每个技术组件为什么选它而不选竞品？
4. 项目需求架构文档（tex-pdf）如何维护和撰写？
5. 后续需求新增如何处理？
6. 文档版本如何迭代？
7. 自闭环机制如何运行？

本文档整合自 AGENT-ARCHITECTURE.md、tech-selection.md、DOC-LIFECYCLE.md、TEAM-MANUAL.md 等多份独立文档，作为唯一的方法论入口。

智能体搭配架构

为什么需要两种智能体

企业级应用需要两类能力：**产品管理**（需求理解、任务拆解、质量审计、进度追踪）和**代码开发**（编写前后端代码、运行命令、构建项目）。

核心分工：魏子政只跟 OpenClaw 对话 → OpenClaw 拆解需求并调度 Cursor → Cursor 写代码 → OpenClaw 审计质量 → 向魏子政汇报。

能力	说明
产品管理	需求理解、任务拆解、质量审计、进度追踪、文档维护、飞书通信
代码开发	编写前后端代码、运行终端命令、理解项目上下文、构建部署

两种智能体的本质区别

维度	OpenClaw (PM)	Cursor (开发)
定位	常驻 AI 助手 + PM	代码编辑器内置 AI
记忆	文件级记忆 (memory/、MEMORY.md)	无跨会话记忆
通信	飞书通道, 可主动推送	无外部通信能力
Skill 来源	~/.openclaw/skills/ + 工作区 skills/	CLAUDE.md + .cursor/rules/*.mdc
Skill 加载	按需 read SKILL.md (AI 自主判断)	自动扫描加载 (alwaysApply / globs)
Skill 格式	普通 Markdown	MDC (YAML front matter + Markdown)
触发方式	魏子政发消息到飞书	OpenClaw 通过 CLI 调度
调用方式	N/A (自己是调度者)	cursor agent -print -trust -workspace < 路径 >
能力边界	不直接改代码 (代码走 Cursor)	不飞书通信、无持久记忆

Skill 加载机制详解

OpenClaw 的 Skill 加载

- OpenClaw 启动会话时, 系统提示中包含 <available_skills> 列表 (每个 skill 的名称和 SKILL.md 路径)
- 收到用户消息后, OpenClaw **根据消息内容自动判断**应该加载哪个 skill
- 用 read 工具读取对应的 SKILL.md 文件内容, 作为后续行动的指令
存放位置:
 - ~/.openclaw/skills/ — 全局 skills (所有项目共享)
 - 工作区 skills/ — 项目专属 skills

Cursor 的规则加载

- CLAUDE.md — 项目根目录, Cursor **每次对话自动加载**, 作为总入口
- .cursor/rules/*.mdc — MDC 格式规则文件, 支持两种触发:
 - alwaysApply: true — 每次对话都加载
 - globs: ["backend/**"] — 只在编辑匹配目录下的文件时加载MDC 文件格式:

```

description: 规则描述
globs: ["backend/**"]      可选
alwaysApply: true          可选
---
# 规则标题
规则内容...

```

本项目中的 Skill/规则清单

文件	给谁	用途
skills/pm/SKILL.md	OpenClaw	需求拆解、任务分配、阶段审计、查证
skills/ui-design/SKILL.md	OpenClaw	设计方案输出、视觉规范把控
skills/tester/SKILL.md	OpenClaw	阶段门控验证、功能测试
CLAUDE.md	Cursor	每次必加载的项目总入口
.cursor/rules/project-overview.mdc	Cursor	技术栈锁定、禁止事项
.cursor/rules/backend-rules.mdc	Cursor	后端编码规范
.cursor/rules/frontend-rules.mdc	Cursor	前端编码规范
.cursor/rules/ui-design.mdc	Cursor	UI 设计规范
.cursor/rules/api-conventions.mdc	Cursor	API 接口规范
.cursor/rules/testing.mdc	Cursor	测试自检要求
.cursor/rules/database.mdc	Cursor	数据库编码约定

关键理解：skills/ 目录下的 SKILL.md 是给 **OpenClaw** 看的（让 PM 知道规范长什么样），.cursor/rules/ 是给 **Cursor** 看的（让开发时自动遵守规范）。两者内容有关联但不相同。

调用链路

一次完整任务执行的完整链路：

1. 魏子政在飞书发消息提出需求
2. **OpenClaw** 加载 pm/SKILL.md，读取 memory 文件恢复上下文，读取项目文档
3. OpenClaw 执行 PM 职责：记录任务 → 拆解需求 → 更新 PROGRESS.md
4. OpenClaw 构造 CLI 命令调度 Cursor：

```

cursor agent --print --trust \
  --workspace <项目路径> "任务描述"

```

5. **Cursor** 自动加载 CLAUDE.md + rules/，读取相关文件，编写代码，返回结果
6. **OpenClaw** 加载 tester/SKILL.md，审计 Cursor 产出（查证三问），如有问题退回修复
7. OpenClaw 向魏子政汇报，更新所有受影响文档

技术选型与理由

锁定技术栈总览

层	技术
前端	Vue 3 + Vite + Element Plus + Pinia + TypeScript
后端	FastAPI + SQLAlchemy (async) + Pydantic
数据库	PostgreSQL 16 (JSONB 存扫描结果)
对象存储	MinIO (S3 兼容)
搜索引擎	OpenSearch 2.x (全文 + 向量)
认证授权	JWT (python-jose + passlib) + RBAC
容器化	Docker + Docker Compose
数据库迁移	Alembic

硬约束：前端不用 Next.js/React，后端不用 Spring/Java。

逐项选型理由

Vue 3 + Element Plus (前端)

- 学习曲线低，模板语法直观，实习生上手快
- Element Plus 企业级组件库开箱即用（表格/表单/弹窗/分页），减少 50%+ UI 工作量
- 国内团队主流，中文文档完善，社区活跃
- TypeScript 一等支持，`<script setup>` + TS 组合式 API
- Vite HMR 毫秒级，开发体验好

不用 Next.js/React：SSR/SSG 对内部管理系统价值不大，多出的复杂度是负担；React 额外状态管理方案 (Redux/Zustand)，Vue 的 Pinia 更简洁。

FastAPI (后端)

- `async/await` 原生支持，天然适合 IO 密集型（上传/搜索/扫描）
- Pydantic 自动生成 Swagger/OpenAPI 文档，前后端对接零成本
- Python 生态与安全扫描工具 (Semgrep/ClamAV)、LLM API 天然互通
- 代码量少，原型快，5-10 人团队产出高

不用 Spring Boot：Java 启动慢、代码量大，与 Python 安全扫描工具集成不便。不用 NestJS：团队主语言 Python，减少跨语言成本。

PostgreSQL 16 (数据库)

- ACID 保证，适合元数据、权限、审计等强一致性数据
- JSONB 类型存储扫描结果（嵌套 JSON），支持查询和索引
- 扩展性强：pgvector、全文检索、窗口函数、CTE

- 运维成熟，免费开源

不用 MongoDB：减少组件数，JSONB 足够。**不用 MySQL：**JSONB 支持不如 PG。

OpenSearch 2.x (搜索)

- 全文 + 向量一体化，kNN 语义搜索
- DSL 支持 per-document 权限查询
- IK Analysis 中文分词
- Apache 2.0 许可（无 SSPL 限制）

不用 Elasticsearch：许可证变更（SSPL），商业使用有合规风险。

MinIO (对象存储)

- S3 API 兼容，随时可切换到 AWS S3/阿里 OSS
- 私有化部署，数据不出域
- 预签名 URL，前端直接下载不经后端转发
- Docker 一容器搞定，自带 Web 管理控制台

JWT + RBAC (认证授权)

- 轻量：Keycloak 需要 JVM + 独立服务，JWT 只需两个 Python 库
- 够用：初期 4 个角色（admin/reviewer/developer/viewer）
- 无状态：Token 自包含，不需要每次查库验证

扩展路径：需要 LDAP/AD/SSO 时再引入 Keycloak，JWT 签发逻辑改对接即可。

Docker Compose (容器化)

- MVP 不需要 K8s，单机部署足够
- `docker compose up` 一键启动所有依赖，新人 5 分钟搭环境
- 5-10 人团队不需要专门 K8s 运维人员

扩展路径：需要多实例/灰度发布时迁移到 K8s（Dockerfile 不改，加 Helm Chart）。

技术栈锁定规则

1. 技术栈一经锁定，OpenClaw 和 Cursor 均不得自行变更
2. 如需变更：魏子政确认 → OpenClaw 评估影响 → 更新文档 → 更新 Cursor rules
3. 新增依赖（npm/pip 包）允许，但不得引入新框架/数据库/中间件
4. 所有变更记录在 tech-selection.md 变更记录中

文档体系与维护

文档目录结构

enterprise-app-store/

根目录（方法论与规范）

AGENT-ARCHITECTURE.md	智能体搭配（详细版）
tech-selection.md	技术选型（详细版）
TEAM-MANUAL.md	团队协作指南
DOC-LIFECYCLE.md	文档生命周期（详细版）
CHANGELOG.md	统一变更日志
PROGRESS.md	进度追踪
CLAUDE.md	Cursor 总入口
BUGS.md	Bug 记录

core/（核心产品文档链）

MRD.tex/pdf	市场需求文档
PRD.tex/pdf	产品需求文档
RFC.tex/pdf	设计方案
TRD.tex/pdf	技术设计文档

demands/（需求文档）

MASTER.tex/pdf	活跃需求 Backlog（持续更新）
DEMAND-xxx.tex/pdf	冻结的需求快照
GUIDE-xxx.tex/pdf	项目指南

reference/（外部研究）

REF-xxx.tex/pdf	外部架构调研
DOC-NAMING-CONVENTION	文档命名规范索引

.cursor/rules/*.mdc

Cursor 编码规则

skills/xxx/SKILL.md

OpenClaw 角色 Skill

文档分类与维护责任

tex-pdf 生成流程

1. 使用 Tectonic LaTeX 引擎: `~/local/tectonic/tectonic input.tex`
2. 必须先 `cd` 到 tex 文件所在目录再执行（相对路径依赖）
3. tex 是源文件（Git 管理），pdf 是交付物（也提交 Git 方便查看）
4. 已知限制：不用 `fontawesome5` 命令、不用 Unicode 箭头（用 `\to`）、`\square` 需数学模式

分类	维护者	更新时机
根目录方法论	OpenClaw	需求变更、技术决策、流程调整
core/ 核心文档	OpenClaw	功能模块变更、架构调整
demands/ 需求	OpenClaw	新需求提出、需求拆解
reference/ 研究	OpenClaw	外部调研完成
Cursor rules	OpenClaw	编码规范变更
代码	Cursor	功能开发、Bug 修复

文档版本管理

采用语义化版本 vX.Y.Z:

数字	含义
X (主版本)	架构重大变更、技术栈变更
Y (次版本)	新增模块/功能、重要规范新增
Z (补丁版本)	文档修正、措辞优化、小补丁

每个重要文档头部包含版本号、日期、更新人、变更摘要。

需求新增处理流程

需求来源

来源	处理方式
魏子政直接提	记录 → 评估 → 拆解 → 排期
阶段审计发现	记录 → 评估 → 排入下阶段
Bug 修复衍生	随 Bug 修复一起处理

需求处理五步流程

第 1 步：需求记录

写入 memory/YYYY-MM-DD.md，包含需求描述、提出者、优先级。复杂需求追问魏子政确认细节。

第 2 步：影响评估 (OpenClaw 执行)

- 技术影响：需要改哪些模块？新增哪些表/接口/页面？
- 文档影响：哪些文档需要更新？
- 进度影响：对当前里程碑有无影响？

第 3 步：方案设计 (复杂需求)

- 新增数据库表? → 更新 SCHEMA
- 新增 API? → 补充 developer SKILL
- 涉及技术栈变更? → **必须魏子政审批**
- 方案输出后给魏子政确认

第 4 步：任务拆解与分配

- 拆解为可独立完成和验证的开发任务
- 写入 PROGRESS.md
- 分配给 Cursor 执行

第 5 步：开发 → 测试 → 审计 → 交付

- Cursor 执行开发
- OpenClaw 独立验收（查证三问）
- 更新所有受影响文档
- 向魏子政汇报

需求优先级

优先级	含义	处理时效
P0	阻塞当前阶段推进	立即处理
P1	核心功能缺陷/缺失	当天处理
P2	重要但不紧急	排入当前/下阶段
P3	锦上添花	有空再做

需求变更控制

小变更（不影响已有模块）：OpenClaw 直接处理，事后汇报。

中变更（影响已有模块但不涉及架构）：OpenClaw 评估 → 简要说明 → 执行。

大变更（涉及架构/技术栈/数据库结构）：OpenClaw 评估 → 详细方案 → **魏子政审批** → 执行。

需求文档维护：MASTER + DEMAND 双层体系

- **MASTER.tex**（活文档）：持续更新的需求 Backlog，所有需求的最新状态
- **DEMAND-xxx.tex**（冻结快照）：每个需求冻结后的完整版本，不再修改
- 新需求提出时：先写入 MASTER（分配编号 D-xxx），开发完成后冻结为 DEMAND-xxx
- MASTER 是唯一权威来源，DEMAND 是历史快照

自闭环机制

什么是自闭环

自闭环 = 从需求提出到交付完成，**所有环节在同一体系内完成**，不需要外部干预。

需求输入（魏子政）

OpenClaw 接收 记录 评估 设计

Cursor 开发 代码产出

OpenClaw 独立验收 查证

文档同步更新 Git commit

交付输出（魏子政）

反馈 回到 OpenClaw 循环

五个闭环

闭环 1：需求闭环

- 输入：魏子政提需求
- 过程：记录 → 评估 → 设计 → 拆解
- 闭合标志：PROGRESS.md 出现新任务，MASTER.tex 更新

闭环 2：开发闭环

- 输入：Cursor 收到任务
- 过程：读 rules/ → 写代码 → 自测
- 闭合标志：npm run build 通过 + 后端启动正常

闭环 3：验收闭环

- 输入：Cursor 产出代码
- 过程：OpenClaw 独立验收（查证三问 + 功能验证）
- 闭合标志：验收清单全部通过

闭环 4：文档闭环

- 输入：代码变更 / 需求变更
- 过程：更新受影响文档 → 生成新 PDF → 记录 CHANGELOG
- 闭合标志：CHANGELOG.md 有新记录，文档版本号已更新

闭环 5：审计闭环

- 输入：验收通过的开发任务
- 过程：查证三问（目标达成？细节硬伤？换位质疑？）
- 闭合标志：查证三问全部通过，Git commit 完成

闭环监控规则

如果任何闭环未闭合，任务不能标记为「已完成」。

检查项	闭合标准
需求闭环	PROGRESS.md 有任务记录 + MASTER.tex 已更新
开发闭环	Cursor 返回成功 + build 通过
验收闭环	验收清单全部通过
文档闭环	受影响文档已更新 + CHANGELOG.md 有记录
审计闭环	查证三问通过 + Git commit 完成

查证三问（每次验收必执行）

1. 目标真的达成了吗？ — 产出物是否完整覆盖原始需求？
2. 细节有硬伤吗？ — 技术细节、数据、路径、命令是否可验证为正确？
3. 如果我是接收者，我会问什么？ — 换位思考，发散性质疑

文档版本迭代

更新触发条件

触发条件	需要更新的文档
技术栈变更	tech-selection.md → 重新生成相关 PDF
新增功能模块	core/ 文档 + demands/ DEMAND-xxx
数据库结构变更	TRD.tex + SCHEMA
需求新增	MASTER.tex + DEMAND-xxx.tex
规范变更	rules/*.mdc + SKILL.md
阶段里程碑完成	core/ 文档版本号更新

文档更新流程

1. 触发条件出现
2. OpenClaw 判断哪些文档需要更新
3. 更新 Markdown/tex 源文件
4. 如果是 tex 文档 → Tectonic 生成新 PDF
5. 更新文档版本号（头部 vX.Y.Z）
6. 记录到 CHANGELOG.md
7. 更新 DOC-NAMING-CONVENTION 索引

8. 更新 PROGRESS.md
9. 向魏子政汇报（变更摘要 + 新文档位置）

Git 提交规范

feat: 新增应用评分功能

fix: 修复登录页 token 过期未跳转的问题

docs: 更新技术选型文档 v2.0

refactor: 重构上传模块为异步

文档变更和对应代码变更放在同一个 commit 或紧邻的 commit，不允许文档与代码不同步。

—

分工矩阵

工作	谁做	怎么做
接收需求	OpenClaw	飞书对话
需求拆解	OpenClaw	读取 pm/SKILL.md
架构设计	OpenClaw	查证技术文档
数据库设计	OpenClaw 设计 → Cursor 实现	SCHEMA + database.mdc
UI 设计	Cursor	根据 ui-design.mdc 规范自主设计实现
写后端代码	Cursor	backend-rules.mdc + api-conventions.mdc
写前端代码	Cursor	frontend-rules.mdc + ui-design.mdc
环境搭建	Cursor	Docker Compose
自验收	Cursor	testing.mdc
独立验收	OpenClaw	查证三问
修 Bug	Cursor + OpenClaw	BUGS.md 记录
写文档	OpenClaw	tex-pdf + Markdown
生成 PDF	OpenClaw	Tectonic
进度管理	OpenClaw	PROGRESS.md
汇报沟通	OpenClaw	飞书 + Edge TTS 语音
需求变更评估	OpenClaw	分析影响 + 更新文档
Git commit	OpenClaw	统一提交

更新记录

版本	日期	变更内容
v1.0	2026-05-06	初始版本，整合方法论总纲